CI/CD ⭐ ⭐ ⭐ — Interview-Ready Q&A

1. What is CI/CD?
CI/CD stands for **Continuous Integration and Continuous Delivery/Deployment**.
It is a software development and DevOps practice used to automate the process from code commit to testing, artifact creation, and deployment.
Developer
↓
Git
↓
Build
↓
Test
↓
Code Quality
↓
Artifact
↓
Deployment
↓
Production
Simple ga gurthupettuko:
CI/CD = **Code → Build → Test → Deploy → Monitor**
Eng: CI/CD is a set of practices and automation processes that continuously build, test, package, and deliver software reliably.

---

2. What is Continuous Integration?
Continuous Integration means developers frequently integrate their code changes into a shared repository.
Whenever code is pushed, the CI pipeline can automatically:

- Checkout code
- Build the application
- Run unit tests
- Perform code quality checks
- Create artifacts

Developer → Git Push → Build → Test → Quality Check
Simple ga gurthupettuko:
CI = **Code frequently merge + automatically build/test**
Eng: Continuous Integration is the practice of frequently integrating code changes and automatically validating them through build and testing processes.

---

3. What is Continuous Delivery?
Continuous Delivery means the application is automatically built, tested, and prepared for release, but Production deployment generally requires a **manual approval or release decision**.
Code
↓
Build
↓
Test
↓
Artifact
↓
Deploy to Staging
↓
Approval
↓
Production
Simple ga gurthupettuko:
Continuous Delivery = **Production ki ready, but approval tarvata deploy**
Eng: Continuous Delivery keeps software in a deployable state through automated build, testing, and release preparation, with Production deployment typically requiring approval.

4. What is Continuous Deployment?
Continuous Deployment means every change that successfully passes the required automated checks can be deployed automatically to Production.
Git Push
↓
Build
↓
Test
↓
Quality
↓
Production
No manual Production approval is required for each deployment if the pipeline is configured for full automation.
Simple ga gurthupettuko:
Continuous Deployment = **Tests pass → Automatic Production deployment**
Eng: Continuous Deployment automatically releases validated changes to Production without a manual approval step for each release.

---

5. Difference between Continuous Integration, Delivery, and Deployment?

Concept	Main Purpose
CI	Build and test code frequently
Continuous Delivery	Keep application ready for release
Continuous Deployment	Automatically deploy validated changes

Simple ga gurthupettuko:
CI → Build + Test
Delivery → Ready to Release
Deployment → Automatically Release
Eng: CI focuses on integrating and validating changes, Continuous Delivery keeps software deployable, and Continuous Deployment automatically releases validated changes.

---

6. What is a CI/CD Pipeline?
A CI/CD pipeline is a sequence of automated stages used to move code from development to deployment.
Typical pipeline:
Git
↓
Checkout
↓
Build
↓
Unit Test
↓
SonarQube
↓
Package
↓
Artifact Repository
↓
Deploy Dev
↓
Deploy QA
↓
Approval
↓
Deploy Production
Simple ga gurthupettuko:
Pipeline = **Software delivery steps ni automate chese flow**
Eng: A CI/CD pipeline automates the stages required to build, test, package, validate, and deploy an application.

---

7. What happens during the Build stage?
During the Build stage, the CI tool compiles or packages the application and creates a deployable output.
For a Java application, for example:
Source Code
↓
Maven Build
↓
JAR/WAR
For a containerized application:
Source Code
↓
Docker Build

↓
Docker Image
Simple ga gurthupettuko:
Build = **Source Code → Deployable Artifact**
Eng: The Build stage converts source code into a deployable artifact such as a JAR, binary, package, or container image.

---

8. What is the Test stage in CI/CD?
The Test stage validates whether the application behaves correctly.
Common tests include:

- Unit tests
- Integration tests
- API tests
- Functional tests
- Security tests

If critical tests fail, the pipeline should normally stop before deployment.
Simple ga gurthupettuko:
Test Fail → **Pipeline Stop**
Eng: The Test stage automatically validates application functionality and prevents defective changes from progressing through the pipeline.

---

9. What is Code Quality in CI/CD?
Code quality checks analyze source code for issues such as:

- Bugs
- Vulnerabilities
- Code smells
- Duplications
- Test coverage

Tools such as SonarQube can be integrated into the pipeline.
Code
↓
Build
↓
SonarQube
↓
Quality Gate
↓
Continue / Fail
Simple ga gurthupettuko:
Code Quality = **Code production ki safe/maintainable ga unda?**
Eng: Code quality analysis identifies bugs, vulnerabilities, maintainability issues, duplication, and other quality problems before deployment.

---

10. What is a Quality Gate?
A Quality Gate is a set of predefined conditions that code must satisfy before the pipeline can continue.
For example:
Coverage ≥ 80%
Critical Vulnerabilities = 0
Quality Rating = A
If the required conditions are not satisfied, the pipeline can fail.
Simple ga gurthupettuko:
Quality Gate = **Production ki velladaniki quality check**
Eng: A Quality Gate defines measurable quality criteria that code must meet before progressing to later stages of the pipeline.

---

11. What is an Artifact?
An artifact is the output produced by the build process that can be stored and used for deployment.
Examples:
Java → JAR/WAR
Node.js → Package
.NET → Build Package
Docker → Container Image
Artifacts are commonly stored in artifact repositories such as Nexus, Artifactory, or container registries.
Simple ga gurthupettuko:
Artifact = **Build ayina deployable output**
Eng: An artifact is a versioned output of the build process that can be stored, promoted, and deployed across environments.

---

12. Why do we need an Artifact Repository?
Instead of rebuilding the application separately for every environment, we build it once and store the resulting artifact.
Example:
Build Once
↓

Artifact Repository
↓
Dev
↓
QA
↓
Production
This ensures the same tested artifact can be promoted across environments.
Simple ga gurthupettuko:
Build Once → Store → Promote Everywhere
Eng: An artifact repository provides centralized storage and versioning for build outputs so the same tested artifact can be promoted across environments.

---

13. Why should we avoid rebuilding the application for each environment?
If we rebuild separately for Dev, QA, and Production, the resulting artifacts may differ.
Instead:
Source Code
↓
One Build
↓
Artifact v1.2
↓
Dev → QA → Production
This improves consistency and makes troubleshooting and rollback easier.
Simple ga gurthupettuko:
One Artifact → All Environments
Eng: Building once and promoting the same artifact across environments improves consistency, traceability, and deployment reliability.

---

14. What is Deployment in CI/CD?
Deployment is the process of releasing the application artifact into an environment.
Typical environments:
Dev
↓
QA
↓
UAT
↓
Production
Depending on the architecture, deployment may target EC2, Kubernetes, ECS, Lambda, or other platforms.
Simple ga gurthupettuko:
Deployment = **Artifact ni environment lo release cheyyadam**
Eng: Deployment is the process of installing or releasing a validated application artifact into a target environment.

---

15. What is a Deployment Strategy?
A deployment strategy defines **how the new application version is released**.
Common strategies include:

- Rolling deployment
- Blue-Green deployment
- Canary deployment
- Recreate deployment

For example, Kubernetes commonly uses rolling updates to gradually replace old Pods with new Pods.
Simple ga gurthupettuko:
Deployment Strategy = **New version ni ela release cheyyali**
Eng: A deployment strategy defines the method used to introduce a new application version while controlling availability, risk, and rollback behavior.

---

16. What is Blue-Green Deployment?
Blue-Green deployment maintains two environments:
Blue → Current Production
Green → New Version
We deploy and test the new version in Green.
After validation, traffic is switched from Blue to Green.
If there is a problem, traffic can be switched back to Blue.
Simple ga gurthupettuko:
Blue = **Old**
Green = **New**
Traffic switch = **Release**
Eng: Blue-Green deployment maintains separate old and new environments and switches traffic to the new environment after validation.

---

17. What is Canary Deployment?
Canary deployment releases the new version to a small percentage of users first.
Example:
100% Traffic
↓
95% Old Version
5% New Version
If the new version performs well, traffic is gradually increased.
5% → 25% → 50% → 100%
Simple ga gurthupettuko:
Canary = **Small traffic first → Verify → Increase**
Eng: Canary deployment gradually exposes a new version to a small portion of traffic before increasing the rollout based on observed health and performance.

---

18. What is Approval in a CI/CD pipeline?
Approval is a manual checkpoint that requires an authorized person to approve deployment before the pipeline proceeds.
For example:
Build
↓
Test
↓
QA
↓
Manual Approval
↓
Production
This is commonly used for Production deployments where organizations require change control.
Simple ga gurthupettuko:
Approval = **Production deployment mundu human confirmation**
Eng: An approval stage provides a controlled manual gate before sensitive deployment stages such as Production.

---

19. Where should we use manual approval?
Manual approval is commonly useful before:
Production Deployment
For example:
Developer
↓
CI
↓
Build
↓
Test
↓
QA
↓
Approval
↓
Production
For lower environments, full automation is often preferred.
Simple ga gurthupettuko:
Dev/QA = **More automation**
Production = **Controlled approval if required**
Eng: Manual approvals are typically used at controlled release points such as Production when organizational change-management requirements demand human authorization.

---

20. What happens if the Build stage fails?
If the Build stage fails, the pipeline should stop.
I would check:

- Build logs
- Compilation errors
- Dependency issues
- Configuration
- Environment variables
- Tool versions
- Repository connectivity

Build Failed
↓
Check Logs
↓
Identify Root Cause

↓
Fix
↓
Re-run Pipeline
Simple ga gurthupettuko:
Build Fail → **Logs → Root Cause → Fix → Re-run**
Eng: If a build fails, I analyze the pipeline logs and build environment, identify the root cause, correct it, and rerun the pipeline.

---

21. What happens if Tests fail?
Normally, the pipeline should stop and prevent the failed build from being promoted.
I would check:

- Test logs
- Failed test cases
- Application changes
- Dependencies
- Test environment
- Test data

Then I fix the issue and rerun the pipeline.
Simple ga gurthupettuko:
Test Fail → **No Deployment**
Eng: A failed test stage should normally block promotion until the failure is understood and resolved.

---

22. What happens if SonarQube Quality Gate fails?
If the pipeline is configured to enforce the Quality Gate, the pipeline should stop before deployment.
For example:
Build
↓
Test
↓
SonarQube
↓
Quality Gate ❌
↓
Pipeline Stop
I would inspect the SonarQube results and fix the reported quality or security issues.
Simple ga gurthupettuko:
Quality Gate Fail → **Pipeline Stop**
Eng: When an enforced Quality Gate fails, the pipeline should prevent the change from progressing until the required quality criteria are satisfied.

---

23. What is Rollback in CI/CD?
Rollback means returning the application to a previous known-good version after a problematic deployment.
Example:
v1.0 → Production
v1.1 → New Deployment
↓
Problem
↓
Rollback
↓
v1.0
The rollback method depends on the deployment platform and strategy.
Simple ga gurthupettuko:
Bad Release → **Previous Stable Version**
Eng: Rollback restores a previously validated application version when a new deployment causes functional, performance, or availability problems.

---

24. How do you perform rollback in a Kubernetes CI/CD pipeline?
Suppose version v2 has an issue.
I can roll back to the previous known-good version using the deployment mechanism, such as Kubernetes Deployment rollout history, or by reverting the GitOps configuration to the previous desired version.
With Argo CD, I can restore the desired Git revision and synchronize it.
v2 → Problem
↓
Previous Version
↓
Git / Deployment Revision
↓
Argo CD
↓

Kubernetes
Simple ga gurthupettuko:
Bad Version → **Previous Version → Sync → Kubernetes**
Eng: In a Kubernetes GitOps environment, I can rollback by restoring a previous known-good deployment configuration and allowing Argo CD to reconcile the cluster to that version.

---

25. How do you secure a CI/CD pipeline?
I would implement:

- Secrets management instead of hardcoding credentials
- Least-privilege IAM permissions
- Protected Git branches
- Secure artifact repositories
- Dependency and security scanning
- Approval controls for Production
- Audit logs
- Restricted pipeline permissions

Secrets should never be stored directly inside source code or pipeline configuration in plain text.
Simple ga gurthupettuko:
CI/CD Security = **Secrets + Least Privilege + Branch Protection + Scanning + Approval**
Eng: I secure CI/CD pipelines using least privilege, protected source control, secure secret management, security scanning, controlled production access, and auditing.

---

26. What is a webhook in CI/CD?
A webhook allows one system to notify another system when an event occurs.
For example:
Developer
↓
Git Push
↓
Webhook
↓
CI Server
↓
Pipeline Start
This allows pipelines to start automatically when code is pushed or a pull request is merged.
Simple ga gurthupettuko:
Webhook = **Event vachindi ani another system ki notification**
Eng: A webhook is an event-driven HTTP notification mechanism commonly used to trigger CI/CD pipelines when source-control events occur.

---

27. What is the difference between CI and CD tools?
A CI tool generally focuses on:
Build
Test
Code Quality
Artifact
CD tools focus more on:
Deployment
Environment Management
Release
Rollback
In real projects, one platform may support both CI and CD, or different tools may be combined.
Simple ga gurthupettuko:
CI = **Build/Test**
CD = **Release/Deploy**
Eng: CI tools primarily automate code validation and artifact creation, while CD tooling automates or controls application delivery and deployment.

---

28. How would you design a Production CI/CD pipeline?
I would design it with separate stages and controlled promotion:
Developer
↓
Git
↓
Build
↓
Unit Tests
↓
SonarQube
↓

Security Scan
↓
Artifact
↓
Dev
↓
QA
↓
Approval
↓
Production
↓
Monitoring
I would also implement automated rollback or a clearly defined rollback procedure.
Simple ga gurthupettuko:
Code → Quality → Artifact → Test → Approval → Production → Monitor
Eng: I would design a Production pipeline with automated validation, immutable/versioned artifacts, controlled environment promotion, security checks, approvals where required, monitoring, and a reliable rollback mechanism.

---

29. Interview Scenario: Production deployment succeeded, but users are getting 500 errors. What will you do?
First, I would treat it as a Production incident and check the application health and monitoring.
I would compare the new release with the previous version and check:
Deployment
↓
Application Logs
↓
Error Rate
↓
Metrics
↓
Dependencies
↓
Recent Changes
If the new release is confirmed as the cause and the impact is significant, I would initiate rollback to the last known-good version according to the release process.
After service restoration, I would investigate the root cause before redeploying the fixed version.
Simple ga gurthupettuko:
500 Errors → **Monitor → Logs → Recent Release → Rollback if needed → RCA**
Eng: I would first restore service safely, verify whether the new release caused the issue, roll back if necessary, and then perform root-cause analysis before redeploying a corrected version.

---

30. Explain a complete real-time CI/CD flow.
Developer code ni Git repository ki push chestadu.
Webhook or pipeline trigger CI process ni start chestundi. CI tool source code checkout chesi build chestundi, tests run chestundi, and SonarQube or security checks perform chestundi.
Build successful ayithe versioned artifact create chesi artifact repository or container registry lo store chestam.
Next deployment stages Dev and QA environments ki artifact ni promote chestayi. Production ki velladaniki manual approval required ayithe approval stage untundi.
Production deployment tarvata monitoring tools application health ni monitor chestayi. Problem vaste previous known-good version ki rollback chestam.

Developer
↓
Git
↓
Webhook / Trigger
↓
Build
↓
Unit Tests
↓
SonarQube / Security Scan → Artifact → Artifact Repository / Registry → Dev → QA → Approval → ProductionCloud / Kubernetes →
→ Monitoring → Rollback if required

Simple ga gurthupettuko:
Git → Build → Test → Quality → Artifact → Dev → QA → Approval → Prod → Monitor → Rollback
Eng: In a typical CI/CD implementation, code changes trigger an automated pipeline that builds and tests the application, performs quality and security checks, creates a versioned artifact, promotes it through environments, obtains Production approval when required, deploys it, monitors the release, and provides a rollback path if problems occur.